Django . Next.js . Flask . React Native . Firebase

Brief Cheatsheet | Muhammad Faizan Yousaf | WAMO Round 2

Framework	Language	Type	Best for
Django	Python	Full-stack web	Full apps with ORM, admin, auth out of the box
Flask	Python	Micro framework	Lightweight APIs, ML model serving, small services
Next.js	JS/React	Full-stack React	SSR, SSG, SEO-friendly React apps, hybrid rendering
React Native	JS/React	Mobile framework	Cross-platform iOS + Android from one codebase
Firebase	Google	Backend-as-service	Auth, real-time DB, hosting - no backend needed

1 - DJANGO

Q: What is Django? What does 'batteries included' mean?

Django is a full Python web framework. Batteries included = ORM, authentication, admin panel, form validation, security - all built in. You don't install separate packages for these basics.

Q: Django MVT architecture - Model, View, Template

Model: defines database schema + ORM. View: business logic, handles request/response. Template: HTML rendered to the user. Different from MVC - Django's View = Controller, Template = View.

```
# models.py - defines DB table
class Post(models.Model):
title = models.CharField(max_length=200)
content = models.TextField()
created = models.DateTimeField(auto_now_add=True)
# views.py - handles request
def post_list(request):
posts = Post.objects.all() # ORM query
return render(request, 'posts.html', {'posts': posts})
# urls.py - routes URL to view
path('posts/', views.post_list, name='post-list')
```

Q: Django ORM - common queries

ORM lets you query the DB using Python - no raw SQL needed. Translates to SQL behind the scenes.

```
Post.objects.all() # SELECT * FROM post
Post.objects.filter(title='Hello') # WHERE title='Hello'
Post.objects.get(id=1) # single record (raises error if missing)
Post.objects.exclude(title='Draft') # NOT WHERE
Post.objects.order_by('-created') # ORDER BY created DESC
Post.objects.create(title='Hi', content='...') # INSERT
post = Post.objects.get(id=1)
post.title = 'Updated'; post.save() # UPDATE
post.delete() # DELETE
```

Q: Django REST Framework (DRF) - quick setup

DRF adds serializers and API views on top of Django. Serializer converts model to JSON. ViewSet handles all CRUD automatically.

```
# serializers.py
class PostSerializer(serializers.ModelSerializer):
class Meta:
model = Post
fields = ['id', 'title', 'content', 'created']
# views.py - one class handles GET, POST, PUT, DELETE
class PostViewSet(viewsets.ModelViewSet):
queryset = Post.objects.all()
serializer_class = PostSerializer
# urls.py
router = DefaultRouter()
router.register('posts', PostViewSet)
urlpatterns = [path('api/', include(router.urls))]
```

Note: ModelViewSet auto-generates: GET /posts, POST /posts, GET /posts/1, PUT /posts/1, DELETE /posts/1

Q: Django migrations - what are they?

Migrations track changes to your models and apply them to the database. Never edit DB manually - always use migrations.

```
python manage.py makemigrations # detect model changes - create migration file
python manage.py migrate # apply migrations to DB
python manage.py showmigrations # see all migrations + status
```

Q: Django middleware - what is it?

Functions that process every request before it reaches the view and every response before it leaves. Used for auth, CSRF protection, session management, logging.

Note: Django has built-in middleware for CSRF, Sessions, Auth. You can write custom middleware for logging, rate limiting, etc.

---

2 - FLASK

Q: What is Flask? How is it different from Django?

Flask is a micro framework - gives you routing and request/response. Nothing else by default. You choose your ORM, auth, validation. Django gives everything. Flask gives nothing - you build what you need.

Feature	Flask	Django
Size	Micro - minimal	Full-featured - batteries included
ORM	You choose (SQLAlchemy)	Built-in Django ORM
Admin panel	Not included	Built-in at /admin
Auth	Flask-Login (install it)	Built-in
Best for	APIs, ML serving, microservices	Full web apps, content sites
Learning curve	Lower	Higher but more structured

Q: Flask basic app + routes

Decorator-based routing. @app.route maps URL to function.

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route('/predict', methods=['POST'])
def predict():
data = request.json # get JSON body
text = data.get('text', '')
result = model.predict([text]) # your ML model
return jsonify({'prediction': result[0]})
@app.route('/health', methods=['GET'])
def health():
return jsonify({'status': 'ok'})
if __name__ == '__main__':
app.run(debug=True, port=5000)
```

Q: Flask for ML model serving - pattern

Load model once at startup. Expose prediction endpoint. Return JSON response.

```
import pickle
from flask import Flask, request, jsonify
app = Flask(__name__)
model = pickle.load(open('model.pkl', 'rb')) # load once at startup
@app.route('/predict', methods=['POST'])
def predict():
features = request.json['features']
pred = model.predict([features])[0]
return jsonify({'prediction': str(pred)})
```

Note: This is the pattern for HealthMate - Flask serving XGBoost predictions.

Q: Flask Blueprints - organizing large apps

Blueprints split a large Flask app into modules - like routers in Express.

```
# auth/routes.py
auth_bp = Blueprint('auth', __name__, url_prefix='/auth')
@auth_bp.route('/login', methods=['POST'])
def login(): ...
# app.py - register blueprint
from auth.routes import auth_bp
app.register_blueprint(auth_bp)
# Now: POST /auth/login works
```

3 - NEXT.JS

Q: What is Next.js? Why use it over plain React?

Next.js is a React framework that adds: server-side rendering (SSR), static site generation (SSG), file-based routing, API routes, image optimization. Plain React is client-side only - bad for SEO. Next.js renders on server = SEO-friendly.

Feature	React (CRA)	Next.js
Rendering	Client-side only (CSR)	SSR, SSG, CSR, ISR - your choice
Routing	React Router (manual)	File-based - create file = create route
API routes	Separate backend needed	Built-in API routes (/pages/api/)
SEO	Poor - JS loaded late	Excellent - pre-rendered HTML
Performance	Slower initial load	Faster - pre-rendered pages
Use when	SPA, dashboard, no SEO	Marketing sites, e-commerce, blogs, full apps

Q: Rendering modes - SSR vs SSG vs CSR vs ISR

SSR: render on server per request - fresh data, slower. SSG: render at build time - fastest, best for content that rarely changes. CSR: render in browser - SPA, no SEO. ISR: SSG but re-generate after X seconds.

```
// SSG - getStaticProps (build time)
export async function getStaticProps() {
const data = await fetchData();
return { props: { data } }; // baked into HTML at build
}
// SSR - getServerSideProps (every request)
export async function getServerSideProps(context) {
const data = await fetchData(context.params.id);
return { props: { data } };
}
// ISR - revalidate after 60 seconds
return { props: { data }, revalidate: 60 };
```

Q: File-based routing in Next.js

Create a file in /pages - automatically becomes a route. No Router config needed.

```
pages/index.js -> /
pages/about.js -> /about
pages/users/index.js -> /users
pages/users/[id].js -> /users/123 (dynamic route)
pages/api/users.js -> /api/users (API endpoint)
// App Router (Next.js 13+) - uses /app directory
app/page.js -> /
app/about/page.js -> /about
app/users/[id]/page.js -> /users/123
```

Q: Next.js API routes - built-in backend

Create files in /pages/api/ - they become API endpoints. No separate Express server needed for simple backends.

```
// pages/api/users.js
export default async function handler(req, res) {
if (req.method === 'GET') {
const users = await User.find({});
return res.status(200).json(users);
}
if (req.method === 'POST') {
const user = await User.create(req.body);
return res.status(201).json(user);
}
}
```

Q: Image optimization and Link in Next.js

Next.js Image component auto-optimizes images (lazy load, resize, WebP). Link component enables client-side navigation without full page reload.

```
import Image from 'next/image';
import Link from 'next/link';
<Link href='/about'>Go to About</Link>
<Image
src='/photo.jpg'
width={500} height={300}
alt='Profile photo'
/>
```

4 - REACT NATIVE (Brief)

Q: What is React Native? How does it differ from React?

React Native builds mobile apps (iOS + Android) using React. Instead of div/span/p, you use View/Text/Image. Compiles to NATIVE mobile components - not a WebView. Same JS/React logic, different UI primitives.

```
// React (web) // React Native (mobile)
<div> <View>
<p>Hello</p> -> <Text>Hello</Text>
<img src='...' /> <Image source={{uri:'...'}} />
<button>Click</button> <TouchableOpacity>
</div> </View>
```

Q: Core React Native components

View: container (like div). Text: display text (ALL text must be in Text). Image: display images. TextInput: input field. ScrollView: scrollable container. FlatList: efficient scrollable list.

```
import { View, Text, TextInput, FlatList, StyleSheet } from 'react-native';
export default function App() {
return (
<View style={styles.container}>
<Text style={styles.title}>Hello Faizan</Text>
<TextInput placeholder='Type here...' />
</View>
);
}
const styles = StyleSheet.create({
container: { flex: 1, padding: 20 },
title: { fontSize: 24, fontWeight: 'bold' }
});
```

Q: Expo vs bare React Native

Expo: managed workflow - easier setup, built-in APIs (camera, GPS, notifications), limited native customization. Bare React Native: full control, requires Android Studio / Xcode setup.

Note: Expo is the right choice for most projects. Only go bare when you need custom native modules.

Q: Navigation in React Native

React Navigation library handles screen navigation. Stack Navigator: push/pop screens like browser history. Tab Navigator: bottom tab bar. Drawer: side menu.

```
import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
const Stack = createStackNavigator();
function App() {
return (
<NavigationContainer>
<Stack.Navigator>
<Stack.Screen name='Home' component={HomeScreen} />
<Stack.Screen name='Profile' component={ProfileScreen} />
</Stack.Navigator>
</NavigationContainer>
);
}
// Navigate between screens
navigation.navigate('Profile');
navigation.goBack();
```

---

5 - FIREBASE (Brief)

Q: What is Firebase? Why use it?

Firebase is Google's Backend-as-a-Service (BaaS). Gives you: Authentication, Firestore (real-time NoSQL DB), Realtime Database, Storage, Hosting - without building your own backend. Fast to prototype, scales automatically.

Firebase Service	What it does	When to use
Authentication	Login with email, Google, GitHub, phone	Any app needing user accounts
Firestore	Real-time NoSQL document database	Chat, live data, collaborative apps
Realtime Database	JSON tree, syncs across clients instantly	Simple real-time data, older projects
Storage	Store files - images, videos, PDFs	User uploads, media
Hosting	Deploy web apps with SSL, CDN	Quick static/React app deployment
Cloud Functions	Run backend code without a server	Triggers, webhooks, scheduled tasks

Q: Firebase Authentication - setup

Enable providers in Firebase console. Client-side SDK handles everything. No backend needed for basic auth.

```
import { getAuth, signInWithEmailAndPassword,
createUserWithEmailAndPassword, signOut } from 'firebase/auth';
const auth = getAuth();
// Register
await createUserWithEmailAndPassword(auth, email, password);
// Login
await signInWithEmailAndPassword(auth, email, password);
// Logout
await signOut(auth);
// Listen to auth state
auth.onAuthStateChanged(user => {
if (user) console.log('Logged in:', user.uid);
else console.log('Logged out');
});
```

Q: Firestore - CRUD operations

Firestore stores data as collections of documents. Real-time listeners update your UI automatically when data changes.

```
import { getFirestore, collection, doc,
addDoc, getDoc, getDocs, updateDoc, deleteDoc,
onSnapshot } from 'firebase/firestore';
const db = getFirestore();
// Create
await addDoc(collection(db, 'users'), { name: 'Faizan', age: 22 });
// Read all
const snap = await getDocs(collection(db, 'users'));
snap.forEach(doc => console.log(doc.id, doc.data()));
// Read one
const docSnap = await getDoc(doc(db, 'users', userId));
// Update
await updateDoc(doc(db, 'users', userId), { age: 23 });
// Delete
await deleteDoc(doc(db, 'users', userId));
// Real-time listener - auto updates UI
onSnapshot(collection(db, 'users'), (snap) => {
const users = snap.docs.map(d => ({ id: d.id, ...d.data() }));
setUsers(users); // React state update
});
```

Q: Firestore vs Realtime Database - which to choose?

Firestore: structured documents, complex queries, offline support - choose for most new projects. Realtime Database: flat JSON tree, simpler, lower latency for basic real-time sync - legacy choice.

Note: If asked, you can say you chose Firebase for rapid development and real-time data sync without managing a backend.

QUICK FIRE COMPARISON - All 5 Together

Question	Answer
Django vs Flask	Django = full kitchen. Flask = empty kitchen, you bring ingredients. Django for big apps, Flask for APIs/ML.
Next.js vs React	React = frontend only. Next.js = React + SSR + routing + API routes. Use Next.js for production.
SSR vs SSG	SSR: fresh data per request. SSG: pre-built at deploy time. ISR: SSG + auto-rebuild after N seconds.
React vs React Native	React = web (div, span). React Native = mobile (View, Text). Same concepts, different components.
Firebase Firestore vs MySQL	Firestore = flexible NoSQL, real-time, no schema. MySQL = structured, relational, ACID transactions.
When to use Firebase	Rapid prototyping, real-time features, no backend team. Avoid for complex business logic.
Django ORM vs raw SQL	ORM: Python objects -> SQL auto-generated. Safer, faster to write. Raw SQL: complex queries ORM cannot
Flask for ML serving	Load model once at startup. Expose /predict endpoint. Return JSON. Lightweight - no Django overhead need
Next.js API routes vs Express	API routes: simple, serverless, built-in. Express: full control, complex middleware, separate server.
Expo vs bare React Native	Expo: fast setup, managed APIs, limited native access. Bare: full control, needs Xcode/Android Studio.
Firebase Auth vs JWT (custom)	Firebase Auth: done for you, multi-provider, secure. Custom JWT: full control, no vendor lock-in.
Django migrations vs manual DB	Always use migrations. Never edit DB schema manually. makemigrations -> migrate.

You've used all of these in real projects. Connect every answer to your actual experience.